

# Federated Risk-Aware Projection

Mattia Sabella

February 2026

## 1 Problem Setting

Let  $\mathcal{K} = \{1, \dots, K\}$  be the set of clients. We train an open-weight LLM with parameters decomposed as

$$\theta = \theta_0 \oplus \phi,$$

where  $\theta_0$  are frozen base model weights and  $\phi$  are trainable PEFT parameters (e.g., LoRA adapter matrices injected into selected layers). Federated learning proceeds over communication rounds  $t = 0, 1, \dots, T - 1$ .

Client  $k$  holds a private dataset  $\mathcal{D}_k$  consisting of samples  $x$  (e.g., token sequences). A deployable local classifier returns a threat probability

$$C_k(x) \in [0, 1],$$

interpreted as the probability that sample  $x$  belongs to threat/dangerous knowledge.

We assume the server has *no access to raw data* and (optionally) uses secure aggregation, so it only receives aggregated updates.

**Training loss.** Let  $\ell(x; \theta_0, \phi)$  denote the language modeling loss for sample  $x$  (e.g., autoregressive negative log-likelihood). The gradient with respect to PEFT params is

$$g(x; \phi) \triangleq \nabla_{\phi} \ell(x; \theta_0, \phi),$$

which is a *vector/tensor* with the same shape as  $\phi$  (concatenated across LoRA modules).

## 2 Classifier-Guided Soft Partitioning

For each sample  $x \in \mathcal{D}_k$ , define soft weights

$$w_k^U(x) \triangleq 1 - C_k(x), \quad w_k^R(x) \triangleq C_k(x).$$

Intuition:  $w_k^U(x)$  measures *utility-likelihood* (safe), while  $w_k^R(x)$  measures *risk-likelihood* (threat).

### 3 Client-Side Dual Updates

At round  $t$ , the server broadcasts current PEFT parameters  $\phi_t$ . Each selected client  $k \in \mathcal{S}_t$  performs local optimization to produce two updates: a *utility update* and a *risk update*. Define the two weighted objectives:

$$\begin{aligned}\mathcal{L}_k^U(\phi; \phi_t) &\triangleq \mathbb{E}_{x \sim \mathcal{D}_k} [w_k^U(x) \ell(x; \theta_0, \phi)], \\ \mathcal{L}_k^R(\phi; \phi_t) &\triangleq \mathbb{E}_{x \sim \mathcal{D}_k} [w_k^R(x) \ell(x; \theta_0, \phi)].\end{aligned}$$

**Local updates.** Using  $E$  local steps and step size  $\eta$ , the client computes:

$$\phi_{k,t}^U \approx \arg \min_{\phi} \mathcal{L}_k^U(\phi; \phi_t), \quad \phi_{k,t}^R \approx \arg \min_{\phi} \mathcal{L}_k^R(\phi; \phi_t),$$

and forms *deltas* in PEFT space:

$$\Delta_{k,t}^U \triangleq \phi_{k,t}^U - \phi_t, \quad \Delta_{k,t}^R \triangleq \phi_{k,t}^R - \phi_t.$$

**Implementation note (single-pass).** In practice, one can avoid two separate trainings by accumulating gradients into two buffers within the same mini-batch stream:

$$g_k^U \leftarrow \sum_{x \in \mathcal{B}} w_k^U(x) g(x; \phi), \quad g_k^R \leftarrow \sum_{x \in \mathcal{B}} w_k^R(x) g(x; \phi),$$

and then applying optimizer steps to construct  $\Delta_{k,t}^U$  and  $\Delta_{k,t}^R$  (with identical optimizer hyperparameters for both heads).

### 4 Server-Data-Free Global Direction Estimation

Because the server holds no WDMP/MLU probe data, it estimates global utility and risk directions purely from the received client updates. Let  $a_{k,t} \geq 0$  be a base aggregation weight (e.g., proportional to local token count  $n_{k,t}$ ). Define aggregated directions:

$$G_t^U \triangleq \sum_{k \in \mathcal{S}_t} a_{k,t} \Delta_{k,t}^U, \quad G_t^R \triangleq \sum_{k \in \mathcal{S}_t} a_{k,t} \Delta_{k,t}^R.$$

Interpretation:  $G_t^U$  approximates the direction induced by safe-likely data across the federation, and  $G_t^R$  approximates the direction induced by threat-likely data.

**Optional secure aggregation.** If using secure aggregation,  $G_t^U$  and  $G_t^R$  are obtained directly as sums without revealing individual client  $\Delta$ 's.

## 5 Risk-Aware Pareto-Constrained Aggregation (Projection)

We now construct a global update that (i) preserves utility progress, and (ii) suppresses components aligned with risk direction.

### 5.1 Inner Product in PEFT Space

For any two PEFT updates  $A, B$  with the same tensor structure as  $\phi$ , define

$$\langle A, B \rangle \triangleq \sum_i \langle A_i, B_i \rangle_F, \quad \|A\| \triangleq \sqrt{\langle A, A \rangle},$$

where  $i$  ranges over LoRA modules and  $\langle \cdot, \cdot \rangle_F$  is the Frobenius inner product.

Define the projection operator onto  $G_t^R$ :

$$\text{Proj}_{G_t^R}(V) \triangleq \begin{cases} \frac{\langle V, G_t^R \rangle}{\|G_t^R\|^2} G_t^R, & \|G_t^R\| > 0, \\ 0, & \text{otherwise.} \end{cases}$$

### 5.2 Risk-Suppressed Utility Update

For each client utility update  $\Delta_{k,t}^U$ , remove its component aligned with  $G_t^R$ :

$$\tilde{\Delta}_{k,t}^U \triangleq \Delta_{k,t}^U - \lambda_t \text{Proj}_{G_t^R}(\Delta_{k,t}^U),$$

where  $\lambda_t \geq 0$  is the *risk-suppression strength*. Larger  $\lambda_t$  enforces stronger WDMP reduction but may reduce MLU.

**Utility gate (optional).** To ensure we only aggregate updates that improve the global utility direction, apply:

$$\hat{\Delta}_{k,t}^U \triangleq \tilde{\Delta}_{k,t}^U \cdot \mathbf{1}[\langle \tilde{\Delta}_{k,t}^U, G_t^U \rangle > 0],$$

or use a smooth gate  $\hat{\Delta}_{k,t}^U = \tilde{\Delta}_{k,t}^U \cdot \sigma(\alpha \langle \tilde{\Delta}_{k,t}^U, G_t^U \rangle)$ .

### 5.3 Utility Salvage from Threat-Labeled Data

We aim to extract *useful* components present in threat-labeled samples while suppressing dangerous knowledge acquisition.

First remove risk-aligned components from the risk update:

$$\tilde{\Delta}_{k,t}^R \triangleq \Delta_{k,t}^R - \mu_t \text{Proj}_{G_t^R}(\Delta_{k,t}^R),$$

with  $\mu_t \geq 0$ .

Then retain only the part aligned with the global utility direction:

$$\hat{\Delta}_{k,t}^R \triangleq \tilde{\Delta}_{k,t}^R \cdot \mathbf{1}[\langle \tilde{\Delta}_{k,t}^R, G_t^U \rangle > 0].$$

Interpretation: keep threat-derived update components that behave like safe-derived capability gains, discard those aligned with risk direction.

## 6 Final Aggregation and Global Update

Define final per-client contribution

$$U_{k,t} \triangleq \widehat{\Delta}_{k,t}^U, \quad R_{k,t} \triangleq \widehat{\Delta}_{k,t}^R.$$

Aggregate with weights  $a_{k,t}$  and a salvage coefficient  $\gamma_t \geq 0$ :

$$\Delta_t \triangleq \sum_{k \in \mathcal{S}_t} a_{k,t} U_{k,t} + \gamma_t \sum_{k \in \mathcal{S}_t} a_{k,t} R_{k,t}.$$

Update global PEFT parameters:

$$\phi_{t+1} \triangleq \phi_t + \Delta_t.$$

Typical schedule: start with small  $\gamma_t$  (conservative salvage), then increase as WDMP remains controlled.

## 7 Optional Enhancements (Still Server-Data-Free)

### 7.1 Orthogonality Regularization (Client-Side Disentanglement)

To make utility and risk directions more separable, client  $k$  may jointly train  $(\phi^U, \phi^R)$  with:

$$\min_{\phi^U, \phi^R} \mathcal{L}_k^U(\phi^U; \phi_t) + \mathcal{L}_k^R(\phi^R; \phi_t) + \eta_{\text{ortho}} \cdot \cos(\Delta_{k,t}^U, \Delta_{k,t}^R),$$

where  $\cos(A, B) \triangleq \frac{\langle A, B \rangle}{\|A\| \|B\|}$ . This penalizes overlap and improves projection efficacy.

### 7.2 Scalar-Only Feedback for Robust Weighting

Each client can optionally report two scalars (no data, no logits):

$$u_{k,t} = \Delta \text{MLU}_k, \quad r_{k,t} = \Delta \text{WDMP}_k,$$

computed on client-private held-out sets. The server adjusts weights:

$$a_{k,t} \propto n_{k,t} \cdot \exp(\rho_u u_{k,t} - \rho_r r_{k,t}),$$

downweighting clients that increase risk and upweighting those that improve utility.

### 7.3 Norm Clipping and DP Noise (Privacy/Security)

Clients clip updates to limit leakage/poisoning:

$$\Delta \leftarrow \Delta \cdot \min\left(1, \frac{S}{\|\Delta\|}\right),$$

and (optionally) add Gaussian noise  $\mathcal{N}(0, \sigma^2 I)$  to provide local DP guarantees.

## 8 Algorithm (RAP-FL)

---

### Algorithm 1 Federated Risk-Aware Projection

---

```

1: Initialize global PEFT parameters  $\phi_0$ 
2: for  $t = 0$  to  $T - 1$  do
3:   Server samples clients  $\mathcal{S}_t$  and broadcasts  $\phi_t$ 
4:   for all  $k \in \mathcal{S}_t$  in parallel do
5:     Client computes weights  $w_k^U(x) = 1 - C_k(x)$ ,  $w_k^R(x) = C_k(x)$ 
6:     Client trains PEFT to obtain  $\Delta_{k,t}^U$  minimizing  $\mathcal{L}_k^U$ 
7:     Client trains PEFT to obtain  $\Delta_{k,t}^R$  minimizing  $\mathcal{L}_k^R$ 
8:     (Optional) clip / DP-noise updates
9:     Client sends  $\Delta_{k,t}^U, \Delta_{k,t}^R$  (or secure-agg shares)
10:  end for
11:  Server forms  $G_t^U = \sum_k a_{k,t} \Delta_{k,t}^U$  and  $G_t^R = \sum_k a_{k,t} \Delta_{k,t}^R$ 
12:  for all  $k \in \mathcal{S}_t$  do
13:     $\tilde{\Delta}_{k,t}^U = \Delta_{k,t}^U - \lambda_t \text{Proj}_{G_t^R}(\Delta_{k,t}^U)$ 
14:     $U_{k,t} = \tilde{\Delta}_{k,t}^U \cdot \mathbf{1}[\langle \tilde{\Delta}_{k,t}^U, G_t^U \rangle > 0]$ 
15:     $\tilde{\Delta}_{k,t}^R = \Delta_{k,t}^R - \mu_t \text{Proj}_{G_t^U}(\Delta_{k,t}^R)$ 
16:     $R_{k,t} = \tilde{\Delta}_{k,t}^R \cdot \mathbf{1}[\langle \tilde{\Delta}_{k,t}^R, G_t^U \rangle > 0]$ 
17:  end for
18:   $\Delta_t = \sum_k a_{k,t} U_{k,t} + \gamma_t \sum_k a_{k,t} R_{k,t}$ 
19:   $\phi_{t+1} = \phi_t + \Delta_t$ 
20: end for
21: return  $\phi_T$ 

```

---